

FormID Database Manager

July 21, 2026

GitHub open issues: 0

#2 deferred Plugin List + Game Context

#16 deferred adjacent seams

ADR-0001 preserved

Offline · no scripts · no network

module

seam

→ leakage

deep module

Deepen Plugin List projection ownership

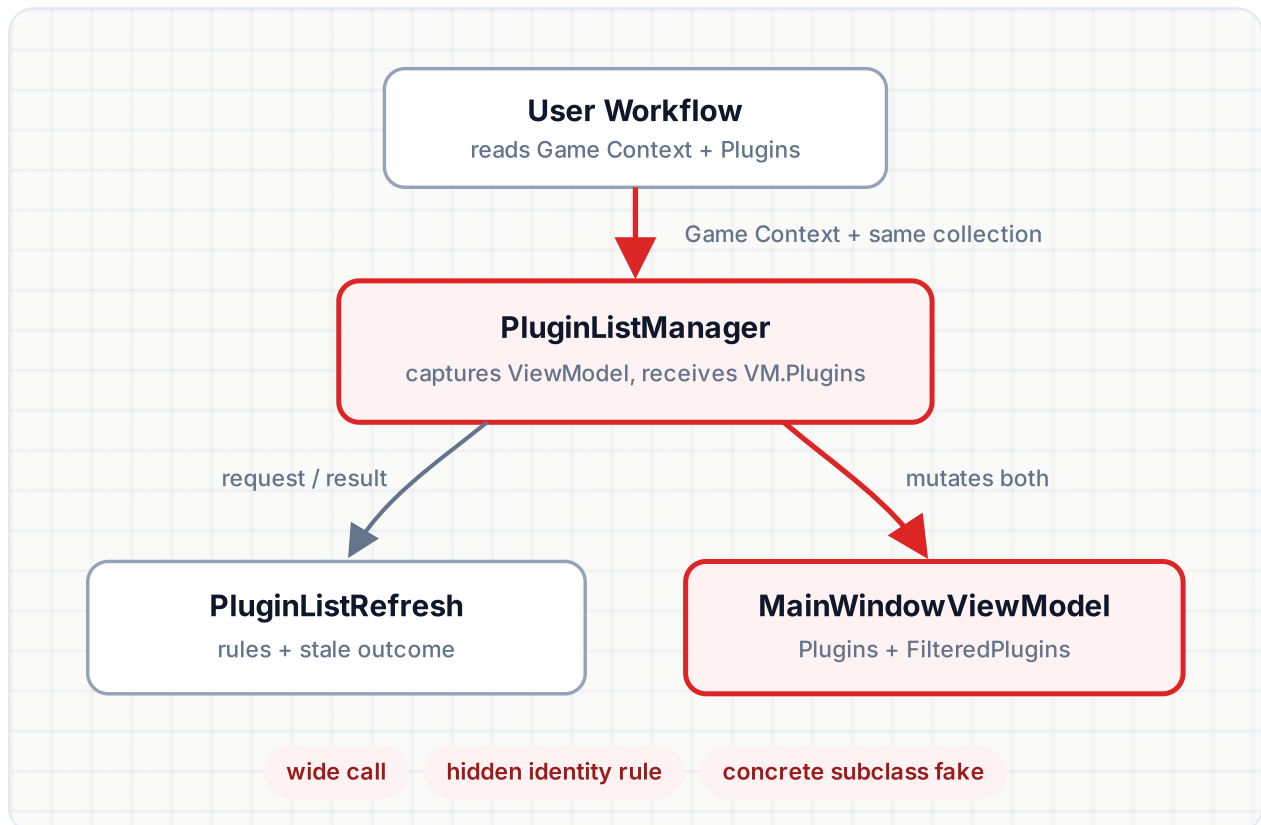
Strong

in-process

Core/Services/PluginListManager.cs:13-218 · Core/Services/PluginListRefresh.cs:5-183 ·
Core/ViewModels/MainWindowViewModel.cs:121-326 · Core/Services/UserWorkflow.cs:284-
301,369-388 ·
Tests/Unit/Services/{PluginListManagerTests,PluginListRefreshTests,UserWorkflowTests}.cs

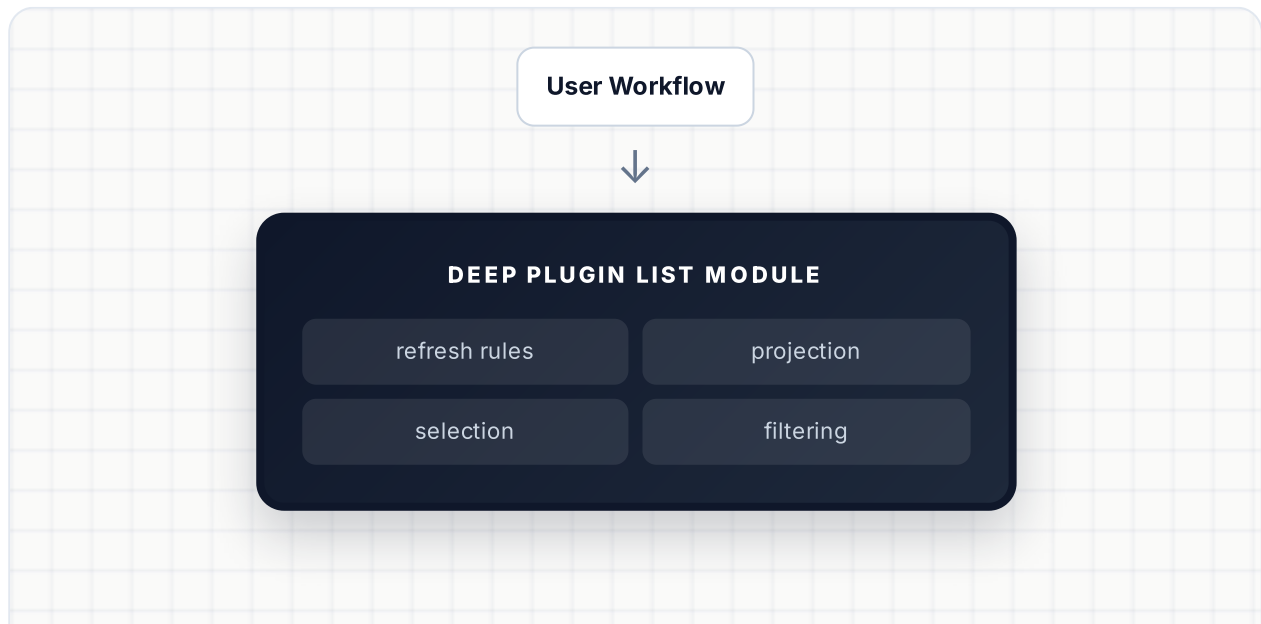
BEFORE · OWNERSHIP LEAKS

same collection crosses seam



AFTER · ONE PROJECTION OWNER

shape deferred



Problem. The Plugin List interface exposes its owned collection and splits refresh, projection, selection, filtering, and freshness knowledge across four modules.

Solution. Concentrate Plugin List projection and selection ownership behind one deep module while retaining load-order and dispatcher adapters as internal seams.

WINS

- locality: one Plugin List owner
- tests hit one interface
- no production subclass fake
- leverage: callers pass intent
- no collection identity rule

Deletion test. Removing the collection hand-off deletes a hidden identity invariant; removing the projection split concentrates its rules in one Plugin List module.

Deepen Game Context at the User Workflow seam

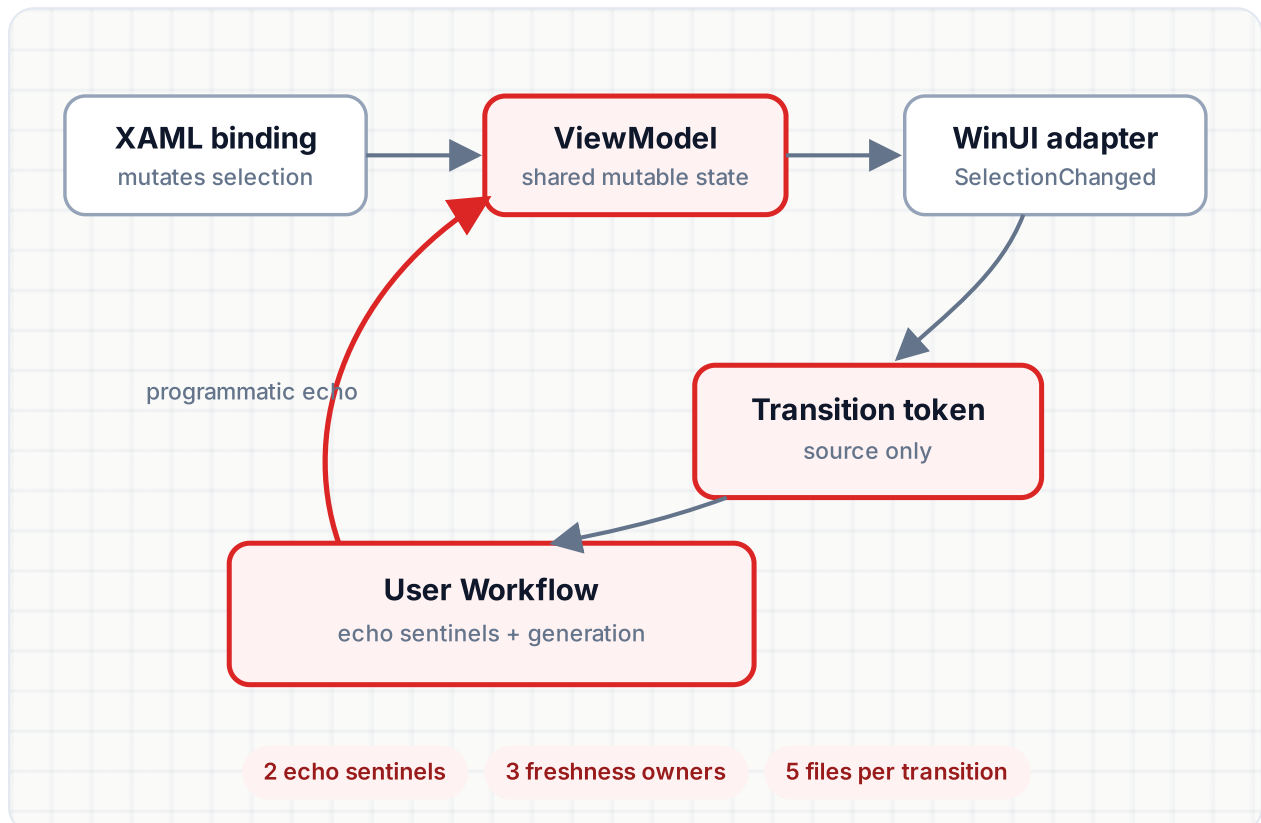
Strong

ports & adapters

WinUI/MainWindow.xaml:40-84 · WinUI/MainWindow.xaml.cs:106-153 ·
Core/Services/GameContextTransition.cs · Core/Services/UserWorkflow.cs:20-22, 55-69, 205-366 · Core/ViewModels/MainWindowViewModel.cs ·
Tests/Unit/Services/UserWorkflowTests.cs:48-175

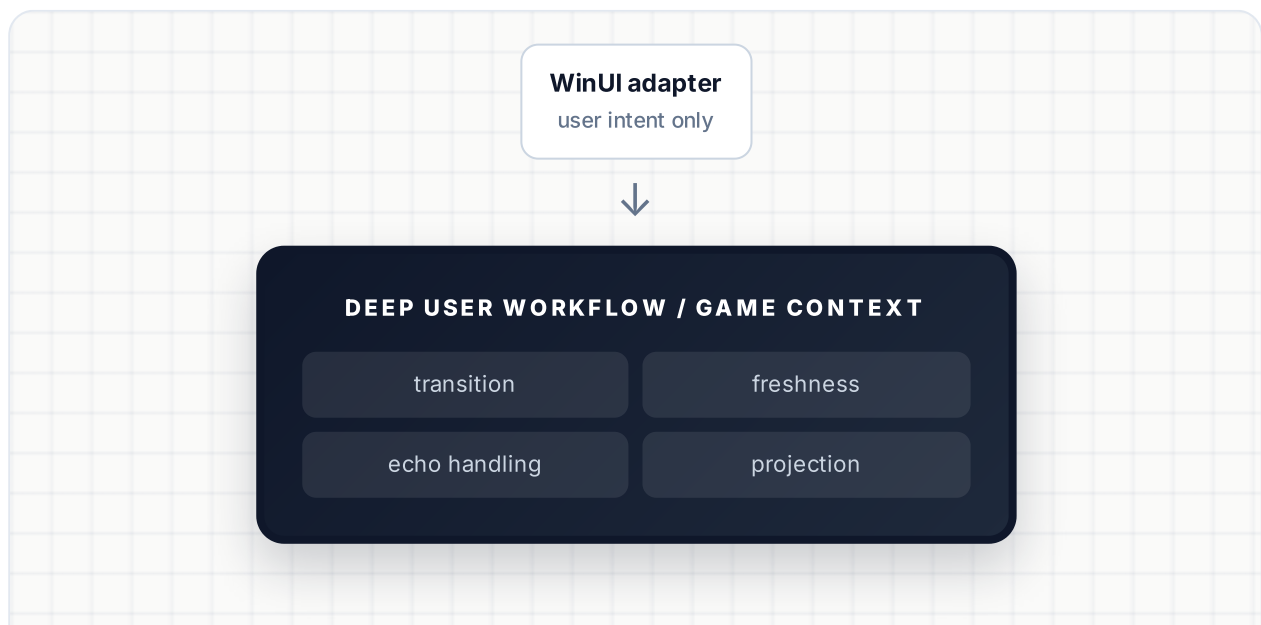
BEFORE · ECHO PROTOCOL

hidden ordering



AFTER · ONE TRANSITION OWNER

WinUI stays thin



Problem. XAML binding order, event causality, programmatic echoes, and latest-wins behavior leak across the Game Context seam.

Solution. Deepen the existing User Workflow so one Game Context transition owner absorbs binding echoes and freshness while WinUI remains an adapter.

WINS

- locality: ordering in one module
- one latest-wins policy
- ViewModel loses echo protocol
- leverage: one transition seam
- tests send real intent
- WinUI adapter stays thin

Deletion test. The source-only transition token vanishes when removed; deleting User Workflow would scatter picker, detection, freshness, and refresh rules across WinUI handlers.

Deepen the Mutagen-facing Plugin read seam

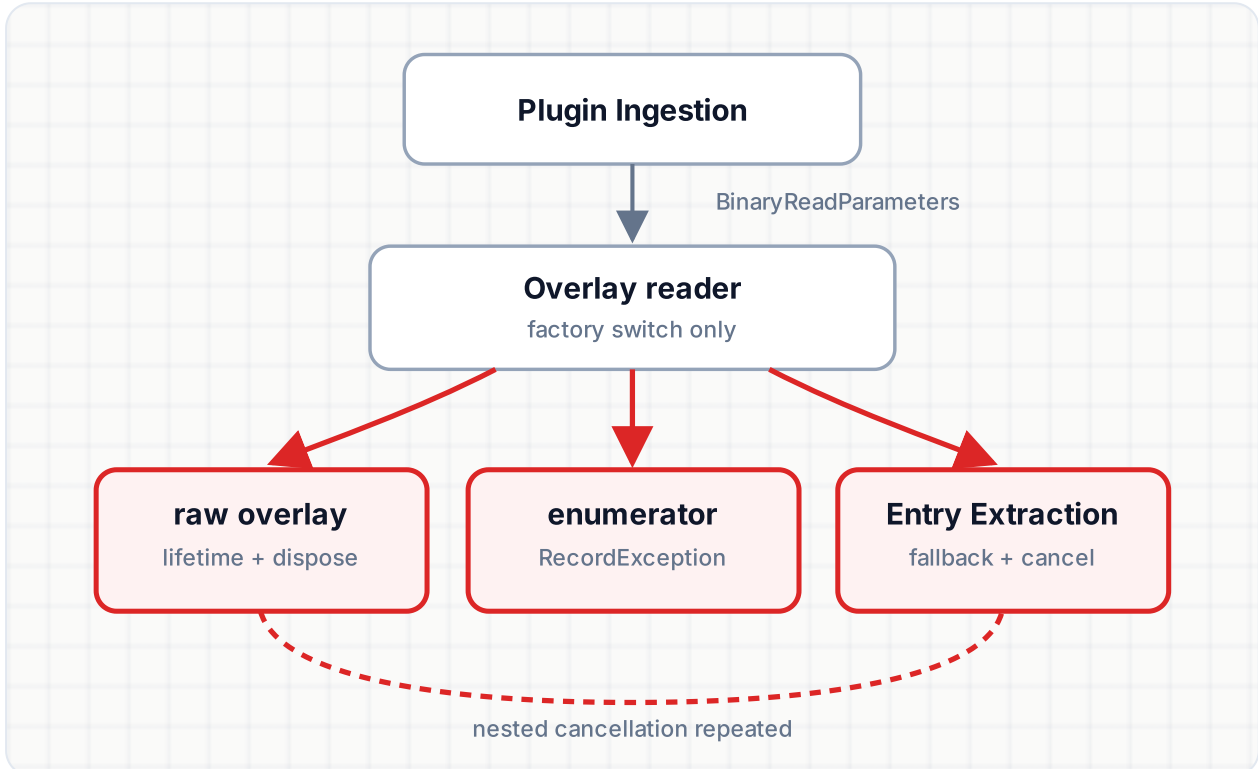
Worth exploring

ports & adapters

Core/Services/IPluginOverlayReader.cs · Core/Services/PluginIngestion.cs:148-220,258-369
· Core/Services/EntryExtraction.cs · Tests/Unit/Services/PluginIngestionTests.cs:910-1182
· Tests/Unit/Architecture/CoreProjectBoundaryTests.cs:139-165

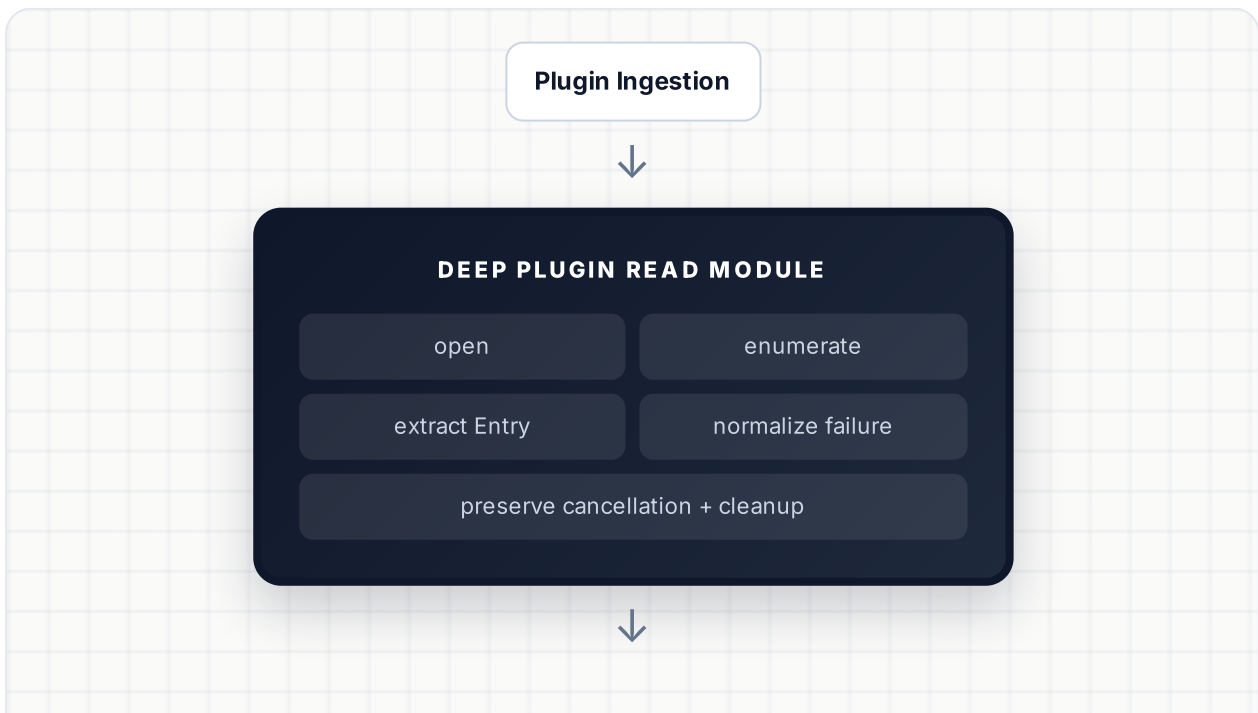
BEFORE · RAW LIFETIME LEAKS

three policies



AFTER · MUTAGEN DETAILS CONTAINED

streaming retained



Problem. The overlay interface hides factory selection but leaks Mutagen lifetime, lazy enumeration, failure phases, cancellation, cleanup, and Entry Extraction.

Solution. Concentrate Plugin opening through Entry Extraction in one Mutagen-facing module while preserving lazy Store consumption and Plugin Ingestion ownership.

WINS

- locality: failure policy once
- overlay lifetime stays hidden
- cancellation remains run control
- leverage: one read seam
- real adapter test surface
- streaming behavior preserved

Deletion test. Removing the current reader moves all lifetime and failure knowledge into one runtime caller; multiple test adapters show the seam is real, so deepen it instead.

Constraint. Keep lazy enumeration inside the Store's atomic Plugin write; current architecture tests pin ownership and will need intent-preserving updates.

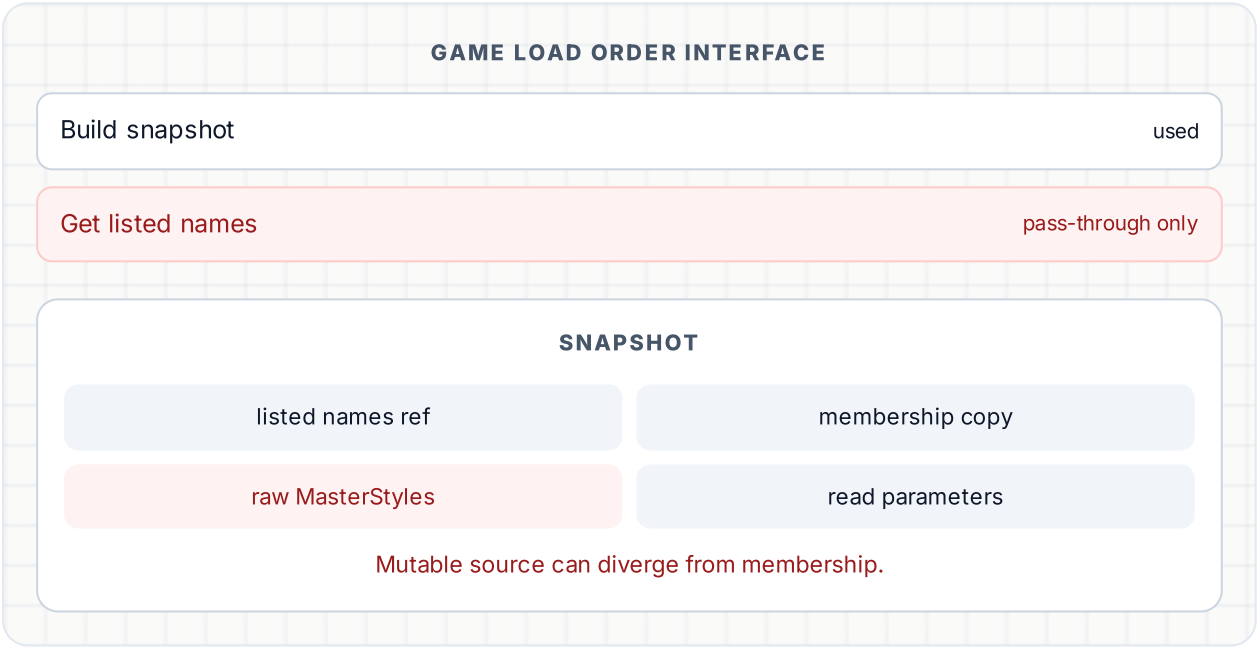
Deepen the Game Load Order snapshot

Strong local-substitutable

```
Core/Services/IGameLoadOrderProvider.cs · Core/Services/GameLoadOrderProvider.cs:38-72 ·
Core/Services/GameLoadOrderSnapshot.cs ·
Tests/Unit/Services/GameLoadOrderProviderTests.cs ·
Tests/Performance/StaticGameLoadOrderProvider.cs
```

BEFORE · SHALLOW SURFACE

test-owned facts



AFTER · OWNED FACTS

smaller interface



Problem. The Game Load Order interface carries a callerless pass-through, exposes test-only Mutagen facts, and lets one snapshot's ordered names diverge from membership.

Solution. Prune unearned surface and make the Game Load Order snapshot own one self-consistent set of load-order facts.

WINS

- interface loses pass-through
- adapters lose boilerplate
- Mutagen facts stay hidden
- leverage: one owned snapshot
- snapshot invariant becomes local
- tests target earned behavior

Deletion test. The pass-through and raw derived fact vanish when removed; deleting the snapshot would spread membership and read-configuration knowledge across two callers.

Contract test-only Store readback surface

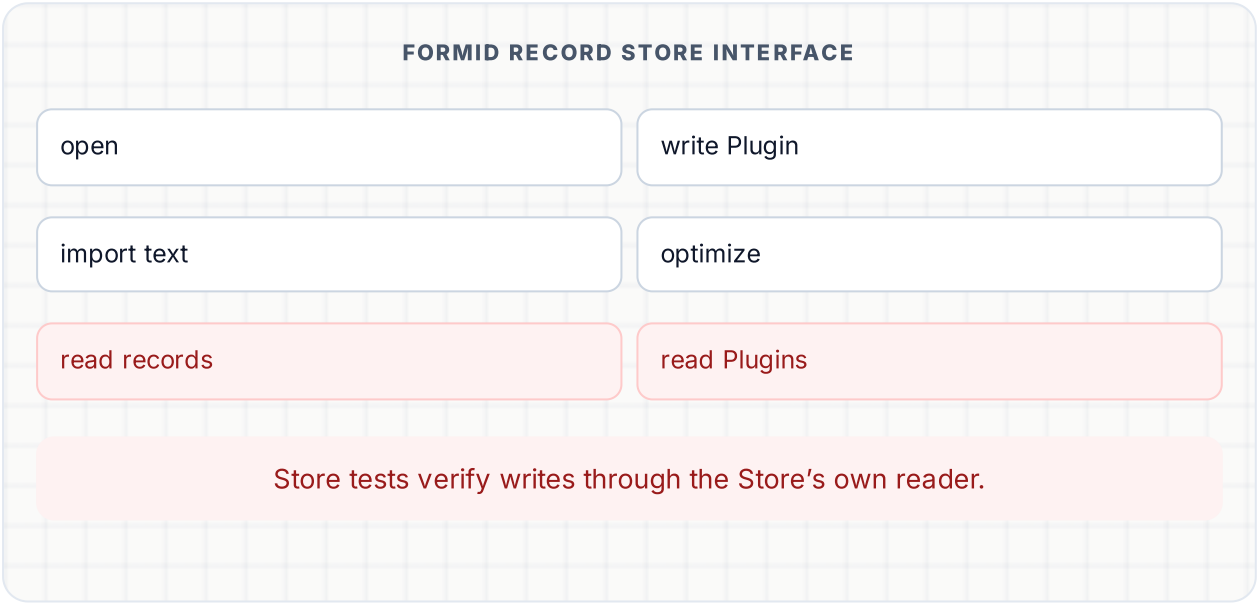
Worth exploring

in-process

```
Core/Services/FormIdRecordStore.cs:37-58,264-327 ·  
Core/Services/FormIdRecordStoreSession.cs ·  
Tests/Unit/Services/FormIdRecordStoreTests.cs:467-518,883-900 ·  
Tests/Integration/ProcessingRunIntegrationTests.cs · docs/adr/0001-formid-record-store-owns-sqlite.md
```

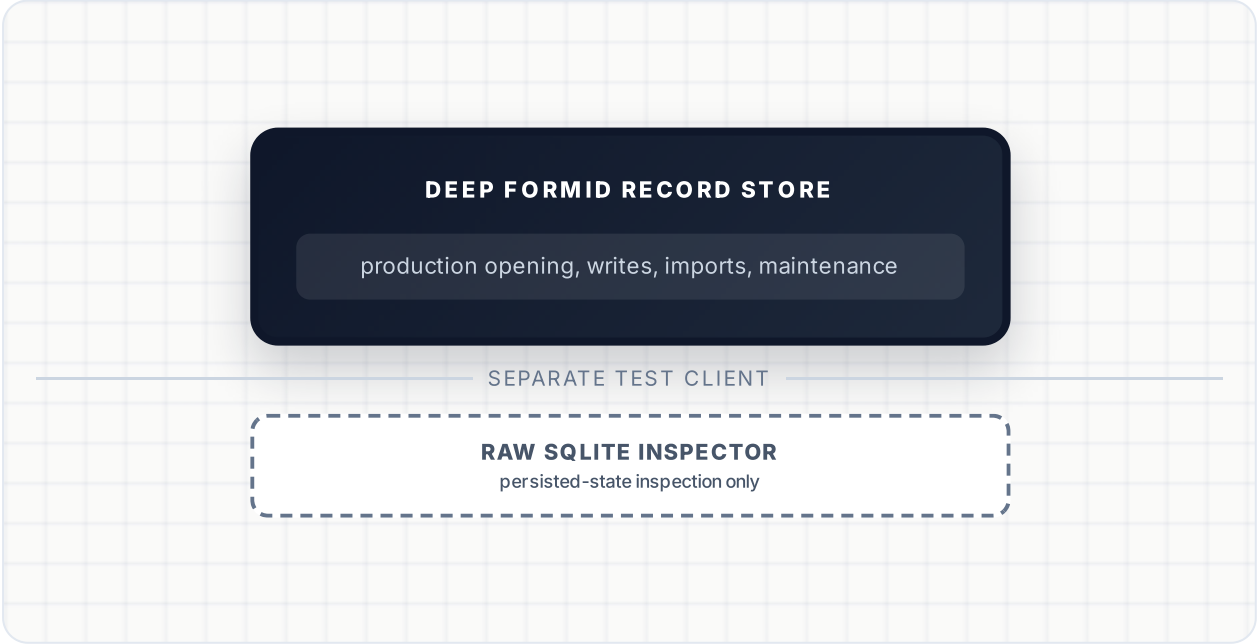
BEFORE · TESTS WIDEN PRODUCTION

0 production readers



AFTER · INDEPENDENT INSPECTION

ADR-aligned



Problem. Two public read operations and their query models have no production caller and make Store tests verify writes through the same implementation.

Solution. Contract the production Store interface and concentrate persisted-state inspection in one test-side raw SQLite client.

WINS

- interface shrinks materially
- tests inspect independently
- ADR seam stays intact
- leverage: Store stays write-focused
- locality: production behavior only
- no alternate setup owner

Deletion test. Removing readback deletes unused production behavior; persisted-state inspection concentrates in the existing test-side SQLite helper.

Decision gate. Retain readback if it is a planned product capability. Today, every caller is a test.

TOP RECOMMENDATION

Deepen Plugin List projection ownership

It removes a hidden collection-identity rule, replaces a concrete inheritance seam, and concentrates a domain module that two completed architecture initiatives explicitly deferred.